ORACLE GOLDENGATE MICROSERVICES

Certificates for a Secure Deployment

TLS • Certificate Trust • Wallets • Distribution Paths • Source/Target Implementation

Client / AdminClient	HTTPS / WSS	GoldenGate MA	Distribution Path	Receiver Service

Secure communications use certificates to establish identity and trust before encrypted traffic is exchanged.

Purpose: a DBA-focused implementation guide for understanding, creating, applying, validating, and troubleshooting certificates in Oracle GoldenGate Microservices Architecture secure deployments.

PREPARED BY
Zubair Mohammed
Principal Database Administrator | High Availability Engineer | Oracle GoldenGate Specialist | Multi-Cloud Architect
Version 1.0 | September 2026

Scope: Oracle GoldenGate Microservices Architecture certificate-based secure deployments. The implementation patterns in this guide are aligned with current Oracle GoldenGate documentation. Commands are examples; use the certificate format, workflow, cryptographic policy, names, paths, validity periods and host names supported by your installed release and enterprise PKI.

1. What Is a Secure GoldenGate Deployment?

In Oracle GoldenGate Microservices Architecture, a secure deployment protects management and data-movement communications with SSL/TLS. Oracle documents secure deployments as using RESTful API calls over SSL/TLS, while secure Distribution Paths use WSS (secure WebSocket) to move trail data between Distribution Service and Receiver Service.

A secure deployment is not just "HTTPS enabled." It combines encrypted transport, server identity, certificate trust, and an authentication method for the Distribution Path.

Secure vs. non-secure

Area	Secure deployment	Non-secure deployment
Management/API	HTTPS / TLS	HTTP
Distribution Path	WSS available/recommended	WS
Certificates	Server/client/trusted chain as required	Not required for plain HTTP
Identity assurance	Certificate trust plus configured authentication	No TLS certificate validation

Where certificates participate

- Browser/AdminClient to GoldenGate services: the client validates the GoldenGate server certificate and its signing CA.
- Distribution Service to Receiver Service: WSS protects the network path; certificate trust is required between endpoints.
- External identity providers: the IDP server CA certificate may need to be uploaded as a trusted certificate.
- Reverse proxy scenarios: the Distribution Service must trust the NGINX certificate when NGINX terminates the secure endpoint.

What TLS provides

Security property	Meaning for GoldenGate
Authentication	The endpoint proves its identity using a certificate and private key.
Confidentiality	Traffic is encrypted after the TLS handshake establishes session keys.
Integrity	TLS detects tampering of protected network traffic.

2. Certificate Fundamentals for a GoldenGate DBA

An X.509 certificate binds an identity (for example a GoldenGate host name or Distribution client identity) to a public key. The certificate is signed by a Certificate Authority (CA). The matching private key remains protected by the owning endpoint and is not shared.

Private Key (secret)	CSR	CA signs	Certificate	Peer validates

Conceptual certificate lifecycle: generate key -> request identity -> CA signs -> deploy certificate -> peer validates trust.

The certificate objects you must understand

Object	Purpose	Where it matters
Root / trusted CA certificate	Trust anchor used to validate certificates signed by that CA.	Trusted certificate store / wallet on the validating side.
Server certificate	Identifies the GoldenGate service endpoint.	Server wallet; host name must align with certificate identity.
Server private key	Proves possession of the server identity.	Server wallet only; protect it.
Client / Distribution certificate	Identifies the Distribution Service client for certificate authentication.	Distribution Service outbound/client wallet.
CSR	Certificate Signing Request containing identity/public-key information to be signed.	Used during certificate creation; not the runtime credential.

Self-signed / private CA vs. trusted CA

Approach	Advantages	Considerations
Self-signed/private CA	No commercial fee; full control; useful for labs/internal environments.	Trust must be explicitly distributed; browsers/tools may warn until CA is trusted; lifecycle is your responsibility.
Enterprise/commercial CA	Established trust and identity validation; easier client/browser trust.	CA process, cost/policy, renewal and organizational dependencies.

Oracle specifically warns that adding a non-CA self-signed certificate as a trusted certificate in the Certificate Management CA Cert section is not supported. The trusted object must be a CA certificate.

3. How Certificates Secure GoldenGate Communications

Server-authenticated TLS

When a client connects to a secure GoldenGate endpoint, the server presents its certificate. The client validates the certificate against a trusted CA chain and verifies the server identity. After the TLS handshake completes, encrypted application traffic flows over HTTPS or WSS.

Client Hello	Server Certificate	CA / Name Validation	TLS Session Keys	Encrypted HTTPS/WSS

GoldenGate Distribution Service and Receiver Service

In a secure GoldenGate topology, both ends of the Distribution Path participate in TLS trust. The source-side Distribution Service validates the certificate presented by the target endpoint, while the target Receiver Service validates the client identity when certificate authentication/mTLS is configured. Current deployments can manage server, client/Distribution Client, and trusted CA certificates through GoldenGate certificate management; older wallet-based deployments may store the corresponding identities and trust material in Oracle wallets.

The certificates must trust each other. Oracle documents two practical trust models for self-managed certificates:

- Both endpoint certificates are signed by the same trusted root CA.
- The opposite side certificate/CA is added to the local wallet as trusted, as appropriate to the documented topology.

Hostname validation is critical

The host name used by the Distribution Path must match the target server certificate CN or a SAN DNS name. If the name cannot be resolved/routed, fix DNS or the relevant /etc/hosts entry. A certificate can be cryptographically valid and still fail connection validation because the host name is wrong.

Certificate authentication vs. encrypted transport

WSS provides encrypted transport. GoldenGate also supports target authentication choices such as Certificate, USERIDALIAS, and OAuth. With Certificate authentication, the Distribution Path uses trusted CA certificates and the target authentication method is set to Certificate.

4. Creating Certificates for GoldenGate Microservices

GoldenGate certificate deployments use standard X.509 identities and CA trust. Current Oracle documentation demonstrates PEM-based certificate and private-key workflows with OpenSSL, while wallet-based environments can use Oracle PKI tooling. The examples below show both concepts. Use the method supported by your installed GoldenGate release and organizational PKI.

4.1 PEM / OpenSSL pattern for current certificate management

A common current pattern is to create a trusted Root CA, a server certificate with serverAuth and Subject Alternative Names, and a Distribution Client certificate with clientAuth. The server certificate, its private key, and the CA chain are applied to the secure deployment. The Distribution Client certificate/private key can be added when the deployment originates certificate-authenticated Distribution Paths.

Example server certificate extension file:
extendedKeyUsage = serverAuth
subjectAltName = DNS:gg-target,DNS:gg-target.example.com,DNS:localhost,IP:127.0.0.1

Example creation flow:
openssl req -newkey rsa:2048 -nodes -keyout server_key.pem -new -out server.csr
openssl x509 -CAcreateserial -CA rootCA_cert.pem -CAkey rootCA_key.pem -req -in server.csr -extfile server_cert.cnf -days 365 -out server_cert.pem

For production, prefer certificates issued under your enterprise or commercial PKI. Oracle's self-signed OpenSSL examples are suitable for demonstrating the trust model, not as a blanket production PKI recommendation.

4.2 Wallet / Oracle PKI pattern

4.2.1 Create the trusted Root CA

```
mkdir -p ~/wallet_directory

orapki wallet create -wallet ~/wallet_directory/root_ca \
  -auto_login -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/root_ca \
  -dn "CN=RootCA" -addext_basic_cons -pathlen 10 \
  -keysize 2048 -self_signed -validity 7300 \
  -pwd <StrongPassword> -sign_alg sha256

orapki wallet export -wallet ~/wallet_directory/root_ca \
  -dn "CN=RootCA" -cert ~/wallet_directory/rootCA_Cert.pem \
  -pwd <StrongPassword>
```

The exported rootCA_Cert.pem is the trust anchor distributed to wallets that must trust certificates issued by this CA. Protect the Root CA private key and do not distribute it to GoldenGate runtime nodes merely to establish trust.

4.2.2 Create a server wallet and certificate

```
orapki wallet create -wallet ~/wallet_directory/$(hostname) \
  -auto_login -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/$(hostname) \
  -dn "CN=$(hostname)" -keysize 2048 -pwd <StrongPassword>

orapki wallet export -wallet ~/wallet_directory/$(hostname) \
  -dn "CN=$(hostname)" -request ~/wallet_directory/server_req.pem \
  -pwd <StrongPassword>
```

```
orapki cert create -wallet ~/wallet_directory/root_ca \
  -request ~/wallet_directory/server_req.pem \
  -cert ~/wallet_directory/server_Cert.pem \
  -serial_num 20 -validity 375 -sign_alg sha256
```

4.2.3 Complete the server wallet trust chain

```
orapki wallet add -wallet ~/wallet_directory/$(hostname) \
  -trusted_cert -cert ~/wallet_directory/rootCA_Cert.pem \
  -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/$(hostname) \
  -user_cert -cert ~/wallet_directory/server_Cert.pem \
  -pwd <StrongPassword>
```

At this point the server wallet contains the server identity certificate/private key and the trusted CA certificate used to validate the chain.

4.2.4 Create the Distribution Service client certificate

```
orapki wallet create -wallet ~/wallet_directory/dist_client \
  -auto_login -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/dist_client \
  -dn "CN=dist_client" -keysize 2048 -pwd <StrongPassword>

orapki wallet export -wallet ~/wallet_directory/dist_client \
  -dn "CN=dist_client" -request ~/wallet_directory/dist_client_req.pem \
  -pwd <StrongPassword>

orapki cert create -wallet ~/wallet_directory/root_ca \
  -request ~/wallet_directory/dist_client_req.pem \
  -cert ~/wallet_directory/dist_client_Cert.pem \
  -serial_num 30 -validity 375 -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/dist_client \
  -trusted_cert -cert ~/wallet_directory/rootCA_Cert.pem \
  -pwd <StrongPassword>

orapki wallet add -wallet ~/wallet_directory/dist_client \
  -user_cert -cert ~/wallet_directory/dist_client_Cert.pem \
  -pwd <StrongPassword>
```

Use unique serial numbers for certificates. Treat the sample password and validity periods in Oracle examples as examples only; production values should follow your organization's PKI and secret-management policy.

5. Applying Certificates to GoldenGate Deployments

Oracle categorizes certificate scope in three ways: certificates local to one deployment, certificates shared by deployments supervised by the same Service Manager, and external certificates used between different source and target deployments.

Scope	Meaning	Typical use
Deployment-local	Certificate belongs to a specific deployment.	A single secure deployment.
Service Manager shared	Certificate is managed at Service Manager and shared by supervised deployments.	Multiple local deployments using a common trust/certificate set.
External source/target	External Root CA / certificate trust connects different deployments.	Source and target managed independently or using different CAs.

Certificate Management responsibilities

- Import the correct CA certificate into the trust store that validates the remote endpoint.
- Keep the private key only with the endpoint that owns the certificate.
- Ensure the server certificate identity matches the DNS/host name clients actually use.
- Confirm wallet paths in deploymentConfiguration.dat when troubleshooting Distribution/Receiver trust.
- Do not import a non-CA self-signed leaf certificate into the CA Cert section as though it were a CA certificate.

Same Root CA vs. different Root CAs

The simplest two-deployment design is for source and target server/client certificates to chain to the same trusted Root CA. When source and target are independent and use different CAs, Oracle provides an external Root CA pattern: exchange/import the appropriate external CA certificate so each validating side can build trust to the peer certificate.

Source CA	Source Certificates	Trust Exchange	Target Certificates	Target CA

6. Source and Target Implementation - DBA Runbook

6.1 Source GoldenGate deployment

- Confirm the source deployment is configured as secure and identify its server/client wallet locations.
- Ensure the Distribution Service outbound/client wallet contains the client identity certificate when certificate authentication is used.
- Import the target Root CA (or the CA that signed the target endpoint certificate) into the source trust store.
- If NGINX terminates TLS on the target, add the NGINX signing CA/certificate trust required by the documented design to the Distribution Service client wallet.
- Verify that the target host name in the Distribution Path matches the target certificate CN or SAN and resolves to the correct IP.
- Create the Distribution Path with Target Protocol = wss. For certificate-based target authentication, select Certificate and the appropriate certificate identity.
- Start the path and monitor Distribution Service status/logs for TLS, trust, host-name, or authentication errors.

6.2 Target GoldenGate deployment

- Configure the target as a secure deployment with a server certificate and matching private key/wallet.
- Ensure the Receiver Service local wallet contains the target server identity and the appropriate trusted CA chain.
- If using certificate authentication, import the CA that signs the Distribution Client certificate and configure the certificate-authenticated GoldenGate user/authorization expected by the target. For certificate-authenticated paths, the target certificate user identity must correspond to the Distribution Path certificate Subject DN as required by GoldenGate.
- Expose only the required secure Receiver Service/reverse-proxy endpoint through firewall/load-balancer policy.
- Verify that the certificate CN/SAN represents the host name the source will use.
- After starting the source path, verify the incoming Distribution Path in Receiver Service and confirm it remains running.

Target-only deployments do not need a Distribution client certificate merely to receive a path. A DistClient certificate is needed when that deployment itself acts as a source/distributor using certificate-based client identity.

7. End-to-End Certificate Trust Example

Example: SRC-GG sends trail data to TRG-GG using a secure WSS Distribution Path. Both server certificates are signed by the same internal Root CA.

Component	Certificate / trust requirement
SRC-GG server	Server certificate + private key; Root CA trusted.
SRC Distribution Service	Client/DistClient certificate for certificate authentication; trusts target Root CA.
TRG-GG server / Receiver	Server certificate + private key; Root CA trusted.
Source target URI	Uses a host name matching target certificate CN/SAN.
Distribution Path	Target Protocol = wss; Target Authentication Method = Certificate.

Connection sequence

1. Source resolves target	2. WSS/TLS handshake	3. Target cert validated	4. Client identity authenticated	5. Trail streaming begins

If any trust or identity step fails, the path does not progress to secure trail streaming. This is why certificate-chain validation and host-name consistency should be checked before troubleshooting Extract or Replicat logic.

If using USERIDALIAS instead of certificate authentication

Oracle also documents secure WSS paths using USERIDALIAS. In that pattern, create an Operator user on the target, store its credentials as an alias on the source, select WSS, and ensure the target host name still matches the CN/SAN of the target certificate. TLS still protects the transport; USERIDALIAS provides the target authentication method.

8. Certificate Handling When NGINX Is in Front of GoldenGate

When a reverse proxy terminates the secure endpoint, the certificate presented to the source is the reverse-proxy certificate. The source-side Distribution Service must trust the CA chain that validates that certificate.

- Import the NGINX signing CA / trusted certificate into the Distribution Service client wallet as required by the topology.
- Use a routable host name contained in the NGINX certificate CN or SAN.
- If DNS is unavailable, map that certificate host name to the correct proxy IP in /etc/hosts on the Distribution Service host.
- Do not assume the backend GoldenGate service certificate is the certificate the source validates when TLS terminates at NGINX.

Source Distribution	WSS / TLS	NGINX certificate	NGINX routes	Target Receiver

Operational implication

Certificate renewal on the reverse proxy is a GoldenGate connectivity event. Renewing a certificate with a different CA, CN, or SAN can break a previously healthy Distribution Path unless the source trust store and target URI are updated accordingly.

What to validate after certificate renewal

- Certificate validity dates and chain.
- CN/SAN contains the exact target host name.
- Source trusts the issuing CA.
- WSS handshake succeeds.
- Distribution Path reaches RUNNING state.
- Receiver Service sees the incoming path.

9. Certificate Troubleshooting Matrix

Symptom	Likely cause	DBA check
TLS/WSS connection fails immediately	CA not trusted / wrong wallet	Inspect source client wallet and target/local wallet trust chain.
Host-name verification failure	Target URI does not match CN/SAN	Use the certificate DNS name; correct DNS or /etc/hosts.
Certificate rejected in CA Cert UI	Leaf/non-CA self-signed certificate used as CA	Import a real CA certificate with CA basic constraints.
Path authenticates with TLS but authorization fails	Wrong target authentication configuration	Check Certificate vs USERIDALIAS vs OAuth selection and target identity.
Worked before renewal, fails after	CA or identity changed	Compare issuer, CN/SAN, validity and source trust store.
Browser/AdminClient warns about self-signed cert	CA not trusted by client	Import the signing CA into the client trust store or use enterprise CA.
Target path not visible	Handshake/path never established	Check Distribution Service errors, WSS URI, firewall, Receiver Service and certificate trust.

Useful validation commands

```
# Inspect a PEM certificate
openssl x509 -in certificate.pem -noout -subject -issuer -dates -serial

# Check certificate text including SAN/basic constraints
openssl x509 -in certificate.pem -noout -text

# Display Oracle wallet contents
orapki wallet display -wallet <wallet_path> -pwd <password>
```

For live endpoint diagnosis, use the TLS inspection tooling approved by your organization and compare the presented certificate chain, host name, issuer, and validity with the wallet/trust configuration used by GoldenGate.

10. Production Certificate Checklist

Area	Production check
PKI	Enterprise/private CA selected; ownership and renewal process documented.
Private keys	Protected permissions; not copied to peers; not stored in scripts or source control.
Server identity	CN/SAN matches the production DNS name used by clients and Distribution Paths.
Trust chain	Correct Root/intermediate CA certificates installed in each validating wallet/store.
Source	Distribution client/outbound wallet configured; target CA trusted; WSS URI validated.
Target	Server/Receiver wallet configured; source client trust/authorization configured if certificate auth is used.
Reverse proxy	Proxy certificate trusted by source; proxy host name matches CN/SAN.
Expiry	Monitoring/alerts before expiration; renewal tested in lower environment.
Change control	Certificate replacement includes rollback and post-change path validation.
Evidence	Document certificate serial, issuer, subject/SAN, expiration, wallet location and owner.

Minimum post-change validation

- GoldenGate secure web UI/API responds over HTTPS.
- AdminClient connects using the intended secure URL/trust model.
- Distribution Path is RUNNING over WSS.
- Receiver Service shows the incoming path.
- Extract and Replicat remain healthy.
- No new TLS/trust errors appear in GoldenGate service logs.

11. Key Takeaways

- Certificates establish endpoint identity and trust; TLS then protects GoldenGate management and Distribution Path traffic in transit.
- **GoldenGate secure deployments use server, client/Distribution Client, and trusted CA certificates as required by the topology and authentication method.**
- Both Distribution and Receiver sides must be designed as a trust relationship, not as independent certificate installations.
- The target host name must match the certificate CN/SAN. DNS and certificate identity are part of the same operational design.

- A target-only deployment generally does not need a Distribution client certificate unless it also originates paths.
- Certificate lifecycle management - expiry, renewal, CA rotation and rollback - is part of GoldenGate availability engineering.

12. Oracle References

1. Oracle GoldenGate - Secure Deployments Using Certificates
https://docs.oracle.com/en/database/goldengate/core/26/coredoc/certificates-deployment.html

2. Oracle GoldenGate - Apply Certificates to a Deployment
https://docs.oracle.com/en/database/goldengate/core/26/coredoc/deploy-applying-certificates-oracle-goldengate.html

3. Oracle GoldenGate - Apply Certificates to Source and Target Deployments
https://docs.oracle.com/en/database/goldengate/core/26/ggsol/apply-certificates-source-and-target-deployments-extern.html

4. Oracle GoldenGate - Distribution Service Path
https://docs.oracle.com/en/database/goldengate/core/26/coredoc/distribute-distribution-path.html

5. Oracle GoldenGate - Distribution Service Path Configuration
https://docs.oracle.com/en/database/goldengate/core/26/coredoc/distribute-distribution-path.html

6. Oracle GoldenGate - Secure Communication Using TLS and mTLS
https://docs.oracle.com/en/database/goldengate/core/26/coredoc/secure-data-transit.html

7. Oracle GoldenGate - Securing GoldenGate using Certificates (supplemental Oracle technical article)
https://blogs.oracle.com/dataintegration/securing-goldengate-using-certificates

Publication note: This is a community technical reference for Oracle GoldenGate Microservices Architecture. Certificate formats, UI fields, TLS options, and management workflows can evolve; validate the exact requirements of your installed GoldenGate release and current Oracle support guidance before production implementation.